

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1.0"/>
<title>HabitFlow - Build Better Days</title>
<link
href="https://fonts.googleapis.com/css2?family=Clash+Display:wght@400;500;600;700&family=Cabinet+Grotesk:wght@300;400;500;700;800&display=swap" rel="stylesheet"/>
<style>
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

:root {
  --bg: #05050a;
  --surface: #0f0f1a;
  --card: #161625;
  --border: rgba(255,255,255,0.06);
  --border2: rgba(255,255,255,0.12);
  --text: #f4f4ff;
  --muted: #6b6b8f;
  --muted2: #9898b8;

  /* category colors */
  --s1: #00f5a0;
  --s1d: #00b37a;
  --s1bg: rgba(0,245,160,0.08);

  --s2: #4d9fff;
  --s2d: #2277dd;
  --s2bg: rgba(77,159,255,0.08);

  --s3: #ff7043;
  --s3d: #dd4411;
  --s3bg: rgba(255,112,67,0.08);

  --s4: #c77dff;
  --s4d: #9b40ee;
```

```
--s4bg: rgba(199,125,255,0.08);

--accent: #ff7043;
}

html { scroll-behavior: smooth; }

body {
  background: var(--bg);
  color: var(--text);
  font-family: 'Cabinet Grotesk', sans-serif;
  min-height: 100vh;
  overflow-x: hidden;
}

/* subtle grid overlay on the bg */
body::before {
  content: '';
  position: fixed;
  inset: 0;
  background-image:
    linear-gradient(rgba(255,255,255,0.015) 1px, transparent 1px),
    linear-gradient(90deg, rgba(255,255,255,0.015) 1px, transparent 1px);
  background-size: 60px 60px;
  pointer-events: none;
  z-index: 0;
}

/* background blobs */
.blob {
  position: fixed;
  border-radius: 50%;
  filter: blur(140px);
  pointer-events: none;
  z-index: 0;
  animation: blobDrift 20s ease-in-out infinite alternate;
}

.blob1 {
  width: 600px;
  height: 600px;
  background: rgba(0,245,160,0.04);
```

```

top: -200px;
left: -200px;
}

.blob2 {
width: 500px;
height: 500px;
background: rgba(255,112,67,0.05);
bottom: -150px;
right: -150px;
animation-duration: 16s;
animation-direction: alternate-reverse;
}

.blob3 {
width: 400px;
height: 400px;
background: rgba(77,159,255,0.04);
top: 40%;
left: 50%;
animation-duration: 22s;
}

@keyframes blobDrift {
from { transform: translate(0, 0) scale(1); }
to   { transform: translate(80px, 50px) scale(1.2); }
}

.app {
max-width: 1000px;
margin: 0 auto;
padding: 40px 24px 100px;
position: relative;
z-index: 1;
}

/* ---- NAV ---- */
nav {
display: flex;
align-items: center;
justify-content: space-between;
margin-bottom: 60px;

```

```
}

.nav-brand {
  display: flex;
  align-items: center;
  gap: 14px;
}

.brand-icon {
  width: 42px;
  height: 42px;
  border-radius: 14px;
  background: linear-gradient(135deg, var(--s1), var(--s2));
  display: grid;
  place-items: center;
  font-size: 20px;
  box-shadow: 0 0 30px rgba(0,245,160,0.2);
}

.brand-name {
  font-family: 'Clash Display', sans-serif;
  font-size: 20px;
  font-weight: 700;
  letter-spacing: -0.5px;
}

.brand-name em {
  font-style: normal;
  background: linear-gradient(90deg, var(--s1), var(--s2));
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

.nav-date {
  font-size: 13px;
  color: var(--muted);
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 100px;
  padding: 8px 18px;
}
```

```
/* ---- HERO ---- */
.hero {
  text-align: center;
  margin-bottom: 64px;
}

.hero-eyebrow {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 100px;
  padding: 7px 16px;
  font-size: 12px;
  color: var(--muted2);
  letter-spacing: 1.5px;
  text-transform: uppercase;
  margin-bottom: 24px;
}

.hero-eyebrow span {
  width: 6px;
  height: 6px;
  border-radius: 50%;
  background: var(--s1);
  animation: pulse 2s ease-in-out infinite;
}

@keyframes pulse {
  0%, 100% { opacity: 1; transform: scale(1); }
  50%      { opacity: 0.5; transform: scale(0.8); }
}

.hero h1 {
  font-family: 'Clash Display', sans-serif;
  font-size: clamp(42px, 7vw, 72px);
  font-weight: 700;
  line-height: 1.05;
  letter-spacing: -2px;
  margin-bottom: 20px;
}
```

```

}

.hero h1 .line2 {
  display: block;
  background: linear-gradient(135deg, var(--s1) 0%, var(--s2) 50%, var(--s4) 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

.hero-sub {
  font-size: 18px;
  color: var(--muted2);
  max-width: 480px;
  margin: 0 auto 40px;
  line-height: 1.6;
  font-weight: 300;
}

.hero-quote {
  display: inline-block;
  background: var(--surface);
  border: 1px solid var(--border);
  border-left: 3px solid var(--accent);
  border-radius: 0 14px 14px 0;
  padding: 14px 20px;
  font-size: 14px;
  color: var(--muted2);
  font-style: italic;
  max-width: 520px;
  line-height: 1.6;
}

/* ---- STATS ---- */
.stats {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 16px;
  margin-bottom: 48px;
}

.stat {

```

```

background: var(--card);
border: 1px solid var(--border);
border-radius: 20px;
padding: 24px 20px;
position: relative;
overflow: hidden;
transition: border-color 0.3s, transform 0.3s;
}

.stat:hover {
border-color: var(--border2);
transform: translateY(-2px);
}

/* colored top bar per stat */
.stat::before {
content: '';
position: absolute;
top: 0;
left: 0;
right: 0;
height: 2px;
}

.stat:nth-child(1)::before { background: var(--s1); }
.stat:nth-child(2)::before { background: var(--s2); }
.stat:nth-child(3)::before { background: var(--s3); }
.stat:nth-child(4)::before { background: var(--s4); }

.stat-icon { font-size: 22px; margin-bottom: 14px; }
.stat-val {
font-family: 'Clash Display', sans-serif;
font-size: 38px;
font-weight: 700;
letter-spacing: -1.5px;
margin-bottom: 4px;
line-height: 1;
}

.stat-lbl { font-size: 12px; color: var(--muted); text-transform: uppercase;
letter-spacing: 1px; }

/* stat value colors */

```

```
.s1 { color: var(--s1); }
.s2 { color: var(--s2); }
.s3 { color: var(--s3); }
.s4 { color: var(--s4); }

/* ---- ADD HABIT SECTION ---- */
.section {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 24px;
  padding: 32px;
  margin-bottom: 28px;
}

.section-label {
  font-size: 11px;
  text-transform: uppercase;
  letter-spacing: 2px;
  color: var(--muted);
  margin-bottom: 20px;
}

.add-row {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 12px;
  margin-bottom: 20px;
}

.input-wrap { position: relative; }

.input-wrap input {
  width: 100%;
  background: var(--bg);
  border: 1px solid var(--border);
  color: var(--text);
  font-family: 'Cabinet Grotesk', sans-serif;
  font-size: 15px;
  padding: 14px 18px 14px 46px;
  border-radius: 14px;
  outline: none;
  transition: border-color 0.2s, box-shadow 0.2s;
}
```



```
}

.input-wrap input:focus {
  border-color: var(--s1);
  box-shadow: 0 0 0 3px rgba(0,245,160,0.1);
}

.input-wrap input::placeholder { color: var(--muted); }

.input-icon {
  position: absolute;
  left: 16px;
  top: 50%;
  transform: translateY(-50%);
  font-size: 18px;
  color: var(--muted);
}

.cats {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin-bottom: 20px;
}

.cat-btn {
  display: flex;
  align-items: center;
  gap: 6px;
  padding: 8px 16px;
  border-radius: 100px;
  border: 1px solid var(--border);
  background: transparent;
  color: var(--muted);
  font-family: 'Cabinet Grotesk', sans-serif;
  font-size: 13px;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.2s;
}

.cat-btn:hover { border-color: var(--border2); color: var(--text); }
```

```

.cat-btn.active-health { border-color: var(--s1); background: var(--s1bg); color:
var(--s1); }
.cat-btn.active-study { border-color: var(--s2); background: var(--s2bg); color:
var(--s2); }
.cat-btn.active-fitness { border-color: var(--s3); background: var(--s3bg); color:
var(--s3); }
.cat-btn.active-other { border-color: var(--s4); background: var(--s4bg); color:
var(--s4); }

.add-btn {
background: linear-gradient(135deg, var(--s1), var(--s2));
border: none;
border-radius: 14px;
padding: 14px 28px;
color: #000;
font-family: 'Clash Display', sans-serif;
font-weight: 600;
font-size: 15px;
cursor: pointer;
transition: opacity 0.2s, transform 0.15s;
white-space: nowrap;
}

.add-btn:hover { opacity: 0.9; transform: translateY(-1px); }
.add-btn:active { transform: scale(0.97); }

/* ---- TABS ---- */
.tabs {
display: flex;
gap: 2px;
background: var(--surface);
border: 1px solid var(--border);
border-radius: 16px;
padding: 4px;
margin-bottom: 24px;
}

.tab {
flex: 1;
padding: 11px;
border: none;
background: transparent;

```

```
color: var(--muted);
font-family: 'Cabinet Grotesk', sans-serif;
font-size: 14px;
font-weight: 500;
border-radius: 12px;
cursor: pointer;
transition: all 0.2s;
}

.tab.on {
background: var(--card);
color: var(--text);
box-shadow: 0 2px 12px rgba(0,0,0,0.4);
}

/* panels - hidden by default */
.panel {display: none; }

.panel.on {
display: block;
animation: fadeUp 0.3s ease;
}

@keyframes fadeUp {
from { opacity: 0; transform: translateY(8px); }
to   { opacity: 1; transform: translateY(0); }
}

/* ---- HABIT CARDS ---- */
.cat-group-label {
font-size: 11px;
text-transform: uppercase;
letter-spacing: 2px;
color: var(--muted);
margin: 28px 0 14px;
padding-left: 4px;
}

.hcard {
background: var(--card);
border: 1px solid var(--border);
border-radius: 20px;
```

```
padding: 20px 22px;
margin-bottom: 10px;
display: flex;
align-items: center;
gap: 18px;
transition: all 0.25s;
position: relative;
overflow: hidden;
}

/* left accent bar */
.hcard::before {
  content: '';
  position: absolute;
  left: 0;
  top: 0;
  bottom: 0;
  width: 3px;
  border-radius: 3px 0 0 3px;
  opacity: 0;
  transition: opacity 0.3s;
}

.hcard.done::before { opacity: 1; }
.hcard.done {
  border-color: rgba(0,245,160,0.15);
  background: rgba(0,245,160,0.03);
}

.hcard:hover { border-color: var(--border2); transform: translateX(3px); }

/* category bar colors */
.hcard.cat-health::before { background: var(--s1); }
.hcard.cat-study::before { background: var(--s2); }
.hcard.cat-fitness::before { background: var(--s3); }
.hcard.cat-other::before { background: var(--s4); }

.check-ring {
  width: 40px;
  height: 40px;
  flex-shrink: 0;
  border-radius: 50%;
}
```

```
border: 2px solid var(--border2);
background: transparent;
cursor: pointer;
display: grid;
place-items: center;
font-size: 0;
transition: all 0.25s;
position: relative;
}

.check-ring::after {
  content: '';
  width: 14px;
  height: 14px;
  border-radius: 50%;
  background: var(--muted);
  opacity: 0;
  transition: all 0.25s;
}

.check-ring:hover::after { opacity: 0.4; }
.check-ring:hover { border-color: var(--s1); }

.check-ring.done {
  background: var(--s1);
  border-color: var(--s1);
  box-shadow: 0 0 20px rgba(0,245,160,0.3);
}

.check-ring.done::after {
  content: '✓';
  font-size: 18px;
  color: #000;
  opacity: 1;
  background: transparent;
  font-weight: 700;
  font-family: 'Clash Display', sans-serif;
}

.hinfo { flex: 1; min-width: 0; }
.hname {
  font-size: 16px;
```

```
font-weight: 500;
margin-bottom: 6px;
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
}

.hmeta {
display: flex;
align-items: center;
gap: 10px;
flex-wrap: wrap;
}

.tag {
font-size: 11px;
font-weight: 600;
letter-spacing: 0.5px;
text-transform: uppercase;
padding: 3px 10px;
border-radius: 100px;
}

.tag-health { background: var(--s1bg); color: var(--s1); }
.tag-study   { background: var(--s2bg); color: var(--s2); }
.tag-fitness { background: var(--s3bg); color: var(--s3); }
.tag-other   { background: var(--s4bg); color: var(--s4); }

.streak { font-size: 12px; color: var(--muted); display: flex; align-items: center;
gap: 4px; }
.streak.hot { color: var(--s3); font-weight: 600; }

.streak-dots { display: flex; gap: 3px; align-items: center; }
.sdot { width: 6px; height: 6px; border-radius: 50%; background: var(--border2); }
.sdot.filled { background: var(--s3); }

.hright { display: flex; align-items: center; gap: 10px; flex-shrink: 0; }

.del-btn {
background: none;
border: none;
color: var(--muted);
```

```
cursor: pointer;
padding: 8px;
border-radius: 10px;
font-size: 18px;
transition: all 0.2s;
line-height: 1;
}

.del-btn:hover { color: #ff4d4d; background: rgba(255,77,77,0.1); }

/* empty state */
.empty { text-align: center; padding: 60px 20px; color: var(--muted); }
.empty-icon { font-size: 48px; margin-bottom: 16px; opacity: 0.4; }
.empty p { font-size: 15px; }

/* ---- WEEKLY VIEW ---- */
.weekly-wrap {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 24px;
  overflow: hidden;
}

.week-header {
  display: grid;
  grid-template-columns: 1fr repeat(7, 40px);
  gap: 0;
  padding: 16px 24px;
  border-bottom: 1px solid var(--border);
}

.week-header span {
  font-size: 11px;
  text-transform: uppercase;
  letter-spacing: 1px;
  color: var(--muted);
  text-align: center;
}

.week-row {
  display: grid;
  grid-template-columns: 1fr repeat(7, 40px);
```

```
gap: 0;
padding: 14px 24px;
border-bottom: 1px solid var(--border);
align-items: center;
transition: background 0.2s;
}

.week-row:last-child { border-bottom: none; }
.week-row:hover { background: rgba(255,255,255,0.02); }

.week-name {
font-size: 14px;
font-weight: 500;
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
padding-right: 16px;
}

.wdot {
width: 28px;
height: 28px;
border-radius: 50%;
margin: 0 auto;
display: flex;
align-items: center;
justify-content: center;
font-size: 12px;
}

.wdot.y { background: rgba(0,245,160,0.2); color: var(--s1); }
.wdot.n { background: var(--surface); color: var(--muted); font-size: 9px; }

.pct-row {
display: flex;
align-items: center;
justify-content: space-between;
padding: 24px;
border-top: 1px solid var(--border);
}
```



```

.pct-left h3 { font-family: 'Clash Display', sans-serif; font-size: 36px; font-weight:
700; color: var(--s1); }
.pct-left p { font-size: 13px; color: var(--muted); margin-top: 2px; }

.pct-bar-outer {
  flex: 1;
  height: 8px;
  background: var(--surface);
  border-radius: 100px;
  margin: 0 24px;
  overflow: hidden;
}

.pct-bar-inner {
  height: 100%;
  border-radius: 100px;
  background: linear-gradient(90deg, var(--s1), var(--s2));
  transition: width 1.2s cubic-bezier(0.34, 1.56, 0.64, 1);
}

/* ---- REMINDERS ---- */
.remind-section { margin-bottom: 32px; }

.rs-head {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-bottom: 16px;
}

.rs-head h3 { font-family: 'Clash Display', sans-serif; font-size: 18px; font-weight:
600; }

.count-badge { padding: 3px 12px; border-radius: 100px; font-size: 12px; font-weight:
700; }
.badge-red { background: rgba(255,77,77,0.15); color: #ff6b6b; }
.badge-green { background: var(--slbg); color: var(--s1); }

.rcard {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 18px;

```

```

padding: 18px 20px;
margin-bottom: 10px;
display: flex;
align-items: center;
gap: 16px;
transition: border-color 0.2s;
}

.rcard.pending:hover { border-color: rgba(255,112,67,0.3); }

.rcard-icon {
width: 42px;
height: 42px;
border-radius: 14px;
display: grid;
place-items: center;
font-size: 20px;
flex-shrink: 0;
}

.rci-health { background: var(--s1bg); }
.rci-study { background: var(--s2bg); }
.rci-fitness { background: var(--s3bg); }
.rci-other { background: var(--s4bg); }

.rcard-name { font-size: 15px; font-weight: 500; }
.rcard-sub { font-size: 12px; color: var(--s3); margin-top: 3px; }

.all-done {
background: var(--s1bg);
border: 1px solid rgba(0,245,160,0.2);
border-radius: 18px;
padding: 36px;
text-align: center;
}

.all-done-emoji { font-size: 44px; margin-bottom: 14px; }
.all-done p { color: var(--s1); font-size: 16px; font-weight: 500; }

/* ---- TOAST ---- */
.toast {
position: fixed;

```

```

bottom: 36px;
left: 50%;
transform: translateX(-50%) translateY(16px);
background: var(--card);
border: 1px solid var(--border2);
border-radius: 100px;
padding: 13px 28px;
font-size: 14px;
font-weight: 500;
opacity: 0;
transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
pointer-events: none;
z-index: 999;
white-space: nowrap;
box-shadow: 0 8px 32px rgba(0,0,0,0.5);
}

.toast.show {
  opacity: 1;
  transform: translateX(-50%) translateY(0);
}

@keyframes fireShake {
  0%, 100% { transform: rotate(-5deg); }
  50%      { transform: rotate(5deg); }
}

.fire { display: inline-block; animation: fireShake 0.4s ease infinite; }

/* responsive */
@media (max-width: 700px) {
  .stats { grid-template-columns: repeat(2, 1fr); }
  .add-row { grid-template-columns: 1fr; }
  nav { flex-direction: column; align-items: flex-start; gap: 12px; }
  .week-header, .week-row { grid-template-columns: 1fr repeat(7, 32px); padding: 12px 16px; }
}

</style>
</head>
<body>

<div class="blob blob1"></div>

```

```

<div class="blob blob2"></div>
<div class="blob blob3"></div>

<div class="app">

  <!-- nav -->
  <nav>
    <div class="nav-brand">
      <div class="brand-icon">🔥</div>
      <div class="brand-name">Habit<em>Flow</em></div>
    </div>
    <div class="nav-date" id="navDate"></div>
  </nav>

  <!-- hero section -->
  <div class="hero">
    <div class="hero-eyebrow"><span></span>Your daily dashboard</div>
    <h1>Build habits that<span class="line2">actually stick.</span></h1>
    <p class="hero-sub">Track, streak, and ship your way to the life you want – one day
at a time.</p>
    <div class="hero-quote" id="heroQuote"></div>
  </div>

  <!-- stats row -->
  <div class="stats">
    <div class="stat">
      <div class="stat-icon">📅</div>
      <div class="stat-val s1" id="sTotal">0</div>
      <div class="stat-lbl">Total Habits</div>
    </div>
    <div class="stat">
      <div class="stat-icon">✅</div>
      <div class="stat-val s2" id="sDone">0</div>
      <div class="stat-lbl">Done Today</div>
    </div>
    <div class="stat">
      <div class="stat-icon">🔥</div>
      <div class="stat-val s3" id="sBest">0</div>
      <div class="stat-lbl">Best Streak</div>
    </div>
    <div class="stat">
      <div class="stat-icon">📅 17</div>

```

```

    <div class="stat-val s4" id="sWeek">0%</div>
    <div class="stat-lbl">This Week</div>
  </div>
</div>

<!-- add habit -->
<div class="section">
  <div class="section-label">+ Add a new habit</div>
  <div class="add-row">
    <div class="input-wrap">
      <span class="input-icon">✎</span>
      <input
        type="text"
        id="hInput"
        placeholder="e.g. Read for 20 minutes, Drink 8 glasses of water..."
        maxlength="60"
      />
    </div>
    <button class="add-btn" onclick="addHabit()">Add →</button>
  </div>
  <div class="cats" id="catBtns">
    <button class="cat-btn active-health" data-cat="health"
onclick="pickCat(this)">🦋 Health</button>
    <button class="cat-btn" data-cat="study"
onclick="pickCat(this)">📖 Study</button>
    <button class="cat-btn" data-cat="fitness"
onclick="pickCat(this)">💪 Fitness</button>
    <button class="cat-btn" data-cat="other"
onclick="pickCat(this)">★ Other</button>
  </div>
</div>

<!-- tabs -->
<div class="tabs">
  <button class="tab on" onclick="goTab('habits', this)">📋 Habits</button>
  <button class="tab" onclick="goTab('weekly', this)">📊 Weekly</button>
  <button class="tab" onclick="goTab('reminders', this)">🕒 Reminders</button>
</div>

<!-- habits panel -->
<div class="panel on" id="panel-habits">
  <div id="habitList"></div>

```

```

</div>

<!-- weekly panel -->
<div class="panel" id="panel-weekly">
  <div class="weekly-wrap">
    <div class="week-header" id="weekHeader"></div>
    <div id="weekRows"></div>
    <div class="pct-row">
      <div class="pct-left">
        <h3 id="pctNum">0%</h3>
        <p>completion this week</p>
      </div>
      <div class="pct-bar-outer">
        <div class="pct-bar-inner" id="pctBar" style="width: 0%></div>
      </div>
    </div>
  </div>
</div>

<!-- reminders panel -->
<div class="panel" id="panel-reminders">
  <div class="remind-section">
    <div class="rs-head">
      <h3>Still to do</h3>
      <span class="count-badge badge-red" id="rPendingBadge">0</span>
    </div>
    <div id="rPending"></div>
  </div>
  <div class="remind-section">
    <div class="rs-head">
      <h3>Completed today</h3>
      <span class="count-badge badge-green" id="rDoneBadge">0</span>
    </div>
    <div id="rDone"></div>
  </div>
</div>

</div>

<div class="toast" id="toast"></div>

<script>

```

```

// motivational quotes shown randomly in the hero
const QUOTES = [
  "Small daily improvements lead to stunning results. 🚀",
  "You don't have to be great to start, but you have to start to be great.",
  "Discipline is choosing between what you want now and what you want most.",
  "Every expert was once a beginner. Keep going!",
  "Success is the sum of small efforts repeated day in and day out.",
  "The best time to start was yesterday. The next best time is NOW.",
  "Hack Club Horizons SF — you're going to make it! 🗨️",
  "35 hours of building → one ticket to San Francisco. Let's go. 🖥️",
];

// category metadata
const CAT = {
  health: { label: '🧘 Health', tag: 'tag-health', icon: '🧘', rcls: 'rci-health' },
  study: { label: '📖 Study', tag: 'tag-study', icon: '📖', rcls: 'rci-study' },
  fitness: { label: '💪 Fitness', tag: 'tag-fitness', icon: '💪', rcls: 'rci-fitness' },
  other: { label: '★ Other', tag: 'tag-other', icon: '★', rcls: 'rci-other' },
};

// currently selected category when adding a habit
let selCat = 'health';

// helpers
const todayStr = () => new Date().toISOString().slice(0, 10);

function load() {
  try {
    return JSON.parse(localStorage.getItem('hf2') || '{}');
  } catch {
    return {};
  }
}

function save(data) {
  localStorage.setItem('hf2', JSON.stringify(data));
}

```

```

// sanitize html to prevent xss when rendering user input
function esc(s) {
  return s
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;');
}

// count consecutive days ending today
function streak(dates) {
  if (!dates.length) return 0;

  const sorted = [...new Set(dates)].sort().reverse();
  let count = 0;
  let check = new Date();
  check.setHours(0, 0, 0, 0);

  for (const d of sorted) {
    const dt = new Date(d + 'T12:00:00');
    if (dt.getTime() === check.getTime()) {
      count++;
      check.setDate(check.getDate() - 1);
    } else {
      break;
    }
  }

  return count;
}

// get the last 7 days as yyyy-mm-dd strings
function last7() {
  return Array.from({ length: 7 }, (_, i) => {
    const d = new Date();
    d.setDate(d.getDate() - (6 - i));
    return d.toISOString().slice(0, 10);
  });
}

// render the little dot row for streak count

```



```

function streakDots(n) {
  const max = 7;
  const filled = Math.min(n, max);
  let html = '<div class="streak-dots">';
  for (let i = 0; i < max; i++) {
    html += `<div class="sdot${i < filled ? ' filled' : ''}"></div>`;
  }
  return html + '</div>';
}

// init
document.addEventListener('DOMContentLoaded', () => {
  const now = new Date();

  document.getElementById('navDate').textContent = now.toLocaleDateString('en-US', {
    weekday: 'long',
    month: 'long',
    day: 'numeric',
    year: 'numeric',
  });

  document.getElementById('heroQuote').textContent =
    '"' + QUOTES[Math.floor(Math.random() * QUOTES.length)] + '"';

  // allow enter key to add habit
  document.getElementById('hInput').addEventListener('keydown', e => {
    if (e.key === 'Enter') addHabit();
  });

  renderAll();
});

function renderAll() {
  const data = load();
  renderStats(data);
  renderHabits(data);
  renderWeekly(data);
  renderReminders(data);
}

function renderStats(data) {
  const habits = Object.values(data);

```

```

const today = todayStr();
const days = last7();

const total = habits.length;
const done = habits.filter(h => h.dates.includes(today)).length;
const best = habits.reduce((m, h) => Math.max(m, streak(h.dates)), 0);

const possible = habits.length * 7;
const weekDone = habits.reduce(
  (sum, h) => sum + days.filter(dd => h.dates.includes(dd)).length,
  0
);
const pct = possible > 0 ? Math.round(weekDone / possible * 100) : 0;

document.getElementById('sTotal').textContent = total;
document.getElementById('sDone').textContent = done;
document.getElementById('sBest').textContent = best + '🔥';
document.getElementById('sWeek').textContent = pct + '%';
}

function renderHabits(data) {
  const el = document.getElementById('habitList');
  const today = todayStr();

  if (!Object.keys(data).length) {
    el.innerHTML = `
      <div class="empty">
        <div class="empty-icon">🌱</div>
        <p>No habits yet – add your first one above!</p>
      </div>`;
    return;
  }

  // group by category
  const grouped = {};
  for (const [name, info] of Object.entries(data)) {
    const cat = info.cat || 'other';
    if (!grouped[cat]) grouped[cat] = [];
    grouped[cat].push([name, info]);
  }

  let html = '';

```

```

for (const cat of ['health', 'study', 'fitness', 'other']) {
  if (!grouped[cat]) continue;

  html += `
```

```

    '<span>Habit</span>' + dayNames.map(n => `<span>${n}</span>`).join('');

let html    = '';
let total   = 0;
let possible = habits.length * 7;

for (const [name, info] of habits) {
    html += `<div class="week-row"><div class="week-name">${esc(name)}</div>`;

    for (const dd of days) {
        const done = info.dates.includes(dd);
        if (done) total++;
        html += `<div class="wdot ${done ? 'y' : 'n'}">${done ? '✓' : '·'}</div>`;
    }

    html += '</div>';
}

if (!habits.length) {
    html = `<div style="padding: 40px; text-align: center; color: var(--muted)">No
habits yet!</div>`;
}

document.getElementById('weekRows').innerHTML = html;

const pct = possible > 0 ? Math.round(total / possible * 100) : 0;
document.getElementById('pctNum').textContent = pct + '%';

// small delay so the transition actually plays
setTimeout(() => {
    document.getElementById('pctBar').style.width = pct + '%';
}, 100);
}

function renderReminders(data) {
    const today    = todayStr();
    const pending = Object.entries(data).filter(([i]) => !i.dates.includes(today));
    const done     = Object.entries(data).filter(([i]) => i.dates.includes(today));

    document.getElementById('rPendingBadge').textContent = pending.length;
    document.getElementById('rDoneBadge').textContent    = done.length;
}

```

```

if (!pending.length) {
  document.getElementById('rPending').innerHTML = `
    <div class="all-done">
      <div class="all-done-emoji">🎉</div>
      <p>You've completed ALL habits today – legendary!</p>
    </div>`;
} else {
  document.getElementById('rPending').innerHTML = pending.map(([name, info]) => {
    const cat = info.cat || 'other';
    const s    = streak(info.dates);
    return `
      <div class="rcard pending">
        <div class="rcard-icon ${CAT[cat].cls}">${CAT[cat].icon}</div>
        <div>
          <div class="rcard-name">${esc(name)}</div>
          ${s > 0 ? `<div class="rcard-sub">⚠️ Don't break your ${s}-day
streak!</div>` : ''}
        </div>
      </div>`;
    }).join('');
}

if (done.length) {
  document.getElementById('rDone').innerHTML = done.map([name]) => `
    <div class="rcard" style="opacity: 0.55">
      <div class="rcard-icon" style="background: rgba(0,245,160,0.1)">✅</div>
      <div class="rcard-name">${esc(name)}</div>
    </div>`.join('');
} else {
  document.getElementById('rDone').innerHTML =
    `<p style="color: var(--muted); font-size: 14px; padding: 4px">Nothing done yet –
let's go!</p>`;
}
}

function pickCat(el) {
  document.querySelectorAll('.cat-btn').forEach(b => b.className = 'cat-btn');
  selCat = el.dataset.cat;
  el.classList.add('active-' + selCat);
}

function addHabit() {

```

```
const inp = document.getElementById('hInput');
const name = inp.value.trim();

if (!name) {
  showToast('⚠ Enter a habit name first!');
  return;
}

const data = load();

if (data[name]) {
  showToast('That habit already exists!');
  return;
}

data[name] = { cat: selCat, created: todayStr(), dates: [] };
save(data);

inp.value = '';
renderAll();
showToast('✅ Habit added!');
}

function toggleDone(name) {
  const data = load();
  if (!data[name]) return;

  const today = todayStr();
  const idx = data[name].dates.indexOf(today);

  if (idx === -1) {
    data[name].dates.push(today);
    const s = streak(data[name].dates);
    showToast(`🔥 Done! ${s}-day streak!`);
  } else {
    data[name].dates.splice(idx, 1);
    showToast('Unmarked for today');
  }

  save(data);
  renderAll();
}
```

```
function delHabit(name) {
  if (!confirm(`Delete "${name}"? This can't be undone.`)) return;

  const data = load();
  delete data[name];
  save(data);
  renderAll();
  showToast('🗑 Deleted');
}

function goTab(tab, btn) {
  document.querySelectorAll('.panel').forEach(p => p.classList.remove('on'));
  document.querySelectorAll('.tab').forEach(t => t.classList.remove('on'));
  document.getElementById('panel-' + tab).classList.add('on');
  btn.classList.add('on');
  renderAll();
}

function showToast(msg) {
  const t = document.getElementById('toast');
  t.textContent = msg;
  t.classList.add('show');
  setTimeout(() => t.classList.remove('show'), 2600);
}

</script>

</body>
</html>
```